

# Full Network Monitoring Stack

## SNMP / Syslog / Grafana — Verification Report

Author: Michael Chileshe

### 1. Summary

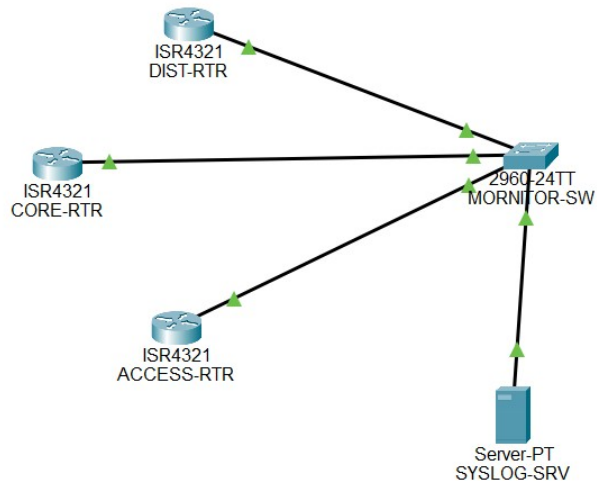
This report documents the build and verification of a complete network monitoring solution across two platforms. The device side (Packet Tracer) proves SNMP community configuration, syslog forwarding, and event collection on Cisco IOS routers. The monitoring side (Docker on WSL) proves a production-style stack: Prometheus scraping SNMP metrics via the SNMP Exporter, visualized on a pre-built Grafana dashboard with live interface status, traffic, and error panels. Both halves verified independently with zero issues.

#### Result overview

| Check                                  | Result | Detail                                            |
|----------------------------------------|--------|---------------------------------------------------|
| SNMP communities configured on routers | PASS   | MONITOR-RO (read), MONITOR-RW (write)             |
| Syslog forwarding to server            | PASS   | Logging to 10.10.10.100, trap level informational |
| Syslog server collects events          | PASS   | Captured interface up/down from CORE-RTR          |
| Docker containers running              | PASS   | 3/3 up: Grafana, Prometheus, SNMP Exporter        |
| Prometheus SNMP target                 | PASS   | UP, scraping 172.20.192.140 every 60s             |
| Grafana dashboard with live data       | PASS   | Interface status, traffic, errors all displaying  |

### 2. Packet Tracer — Network Topology

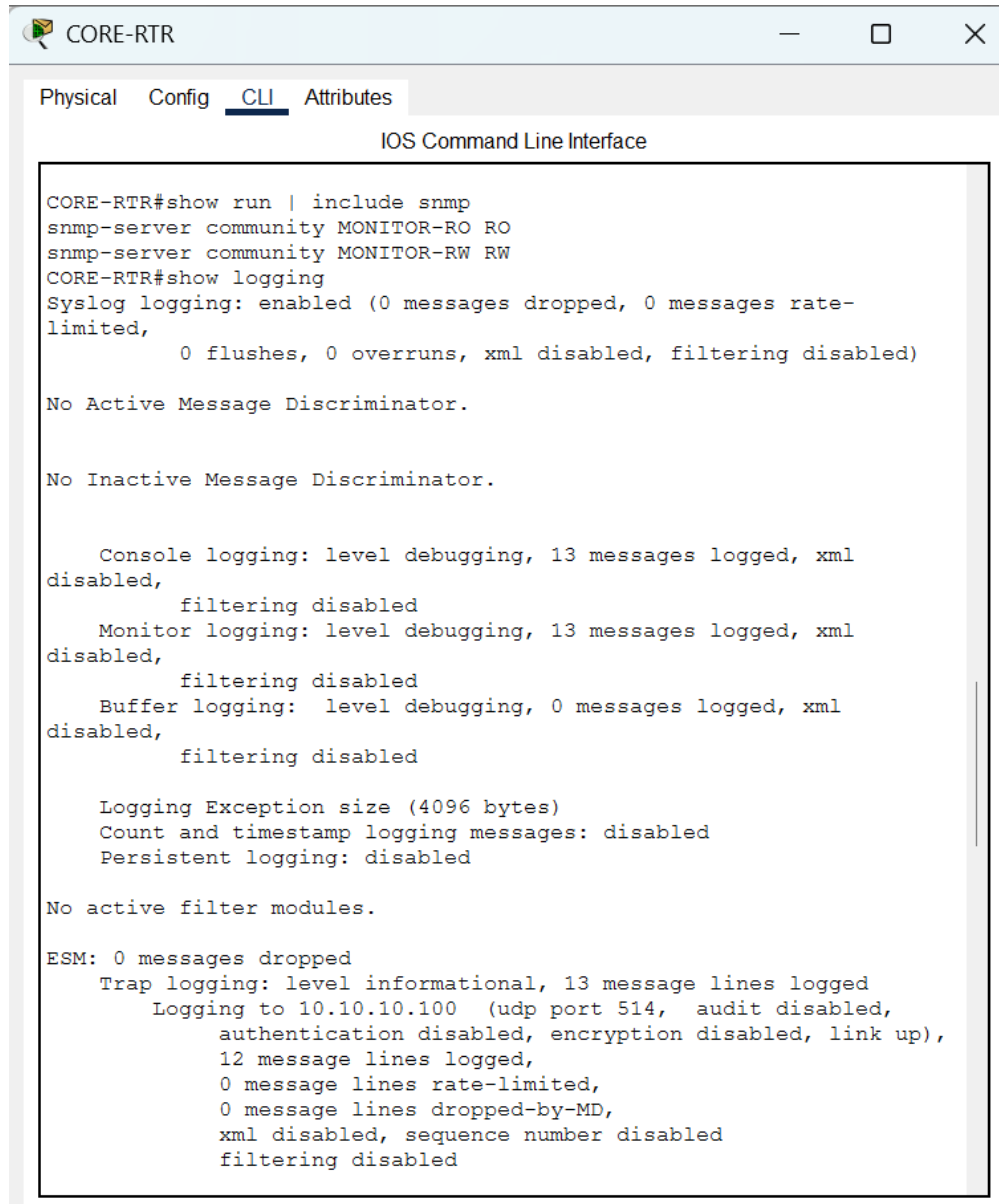
Three ISR 4321 routers (CORE-RTR, DIST-RTR, ACCESS-RTR) connected through a management switch to a syslog server. All routers configured with SNMP communities and syslog forwarding to the central server.



*PT topology: 3 routers, management switch, syslog server*

### 3. SNMP & Syslog Configuration Verified

The SNMP community strings and syslog configuration were confirmed present in the running config via 'show run | include snmp' and 'show logging'. The logging output confirms syslog is forwarding to 10.10.10.100 at the informational trap level, with 12 messages already sent.



```
CORE-RTR#show run | include snmp
snmp-server community MONITOR-RO RO
snmp-server community MONITOR-RW RW
CORE-RTR#show logging
Syslog logging: enabled (0 messages dropped, 0 messages rate-
limited,
    0 flushes, 0 overruns, xml disabled, filtering disabled)
No Active Message Discriminator.
No Inactive Message Discriminator.
Console logging: level debugging, 13 messages logged, xml
disabled,
    filtering disabled
Monitor logging: level debugging, 13 messages logged, xml
disabled,
    filtering disabled
Buffer logging: level debugging, 0 messages logged, xml
disabled,
    filtering disabled
Logging Exception size (4096 bytes)
Count and timestamp logging messages: disabled
Persistent logging: disabled
No active filter modules.
ESM: 0 messages dropped
Trap logging: level informational, 13 message lines logged
Logging to 10.10.10.100 (udp port 514, audit disabled,
authentication disabled, encryption disabled, link up),
12 message lines logged,
0 message lines rate-limited,
0 message lines dropped-by-MD,
xml disabled, sequence number disabled
filtering disabled
```

*CORE-RTR: SNMP communities (MONITOR-RO, MONITOR-RW) confirmed, syslog logging to 10.10.10.100 with 12 messages forwarded*

## 4. Syslog Event Collection

To generate syslog events, CORE-RTR's GigabitEthernet0/0/0 interface was shut down and immediately re-enabled. The router generated timestamped LINK-CHANGED and LINEPROTO-UPDOWN messages, which were forwarded to the syslog server in real time.

## Event generation on CORE-RTR

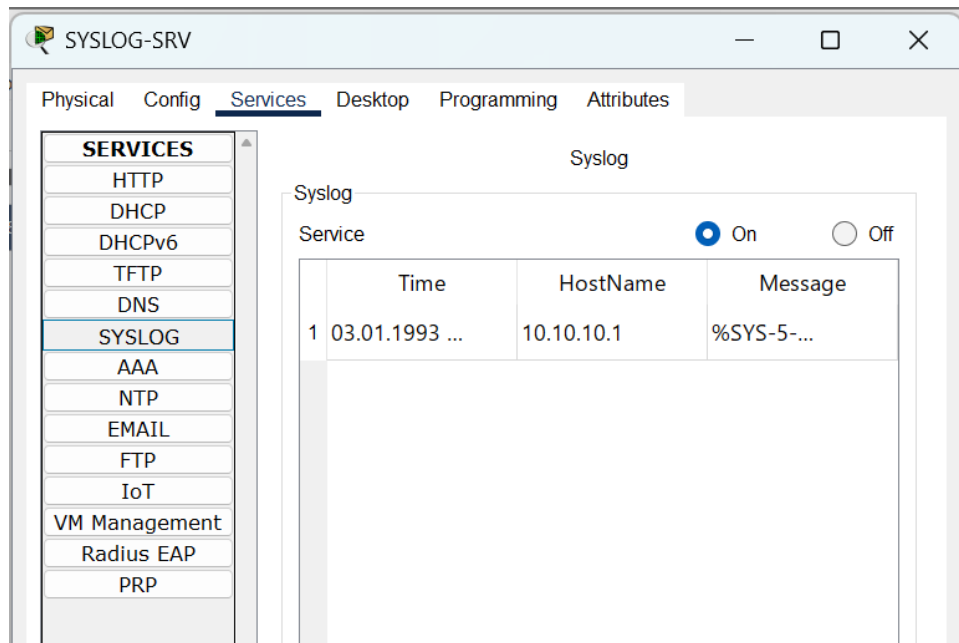
```
CORE-RTR#configure terminal
Enter configuration commands, one per line. End with CNTL/Z.
CORE-RTR(config)#interface GigabitEthernet0/0/0
CORE-RTR(config-if)#shutdown

CORE-RTR(config-if)#no shutdown

CORE-RTR(config-if)#end
CORE-RTR#
*Mar 01, 00:14:35.1414: %LINK-5-CHANGED: Interface
GigabitEthernet0/0/0, changed state to administratively down
*Mar 01, 00:14:35.1414: %LINEPROTO-5-UPDOWN: Line protocol on
Interface GigabitEthernet0/0/0, changed state to down
*Mar 01, 00:14:35.1414: %LINK-5-CHANGED: Interface
GigabitEthernet0/0/0, changed state to up
*Mar 01, 00:14:35.1414: %LINEPROTO-5-UPDOWN: Line protocol on
Interface GigabitEthernet0/0/0, changed state to up
*Mar 01, 00:14:35.1414: SYS-5-CONFIG_I: Configured from console by
console
CORE-RTR#
```

*CORE-RTR: interface shutdown/no shutdown generates linkdown, linkup, and config-change syslog messages*

## Syslog server captures the event



*SYSLOG-SRV: captured the %SYS-5 message from 10.10.10.1 (CORE-RTR) with timestamp*

## 5. Docker Monitoring Stack

Three containers deployed via docker-compose: Prometheus (metrics database), SNMP Exporter (SNMP-to-Prometheus translator), and Grafana (dashboard visualization). All three started cleanly and are running.

## Container deployment

```
michael@Michael:~/Network monitoring stack/docker$ docker compose down
[+] down 4/4
✓Container grafana      Removed      1.1s
✓Container prometheus   Removed      0.7s
✓Container snmp-exporter Removed      0.7s
✓Network docker_default Removed      0.3s
michael@Michael:~/Network monitoring stack/docker$ docker compose up -d
[+] up 4/4
✓Network docker_default Created        0.1s
✓Container grafana      Started       0.8s
✓Container snmp-exporter Started       0.8s
✓Container prometheus   Started       0.7s
michael@Michael:~/Network monitoring stack/docker$
```

*docker compose down / up -d: all containers removed cleanly and restarted, 4/4 operations successful*

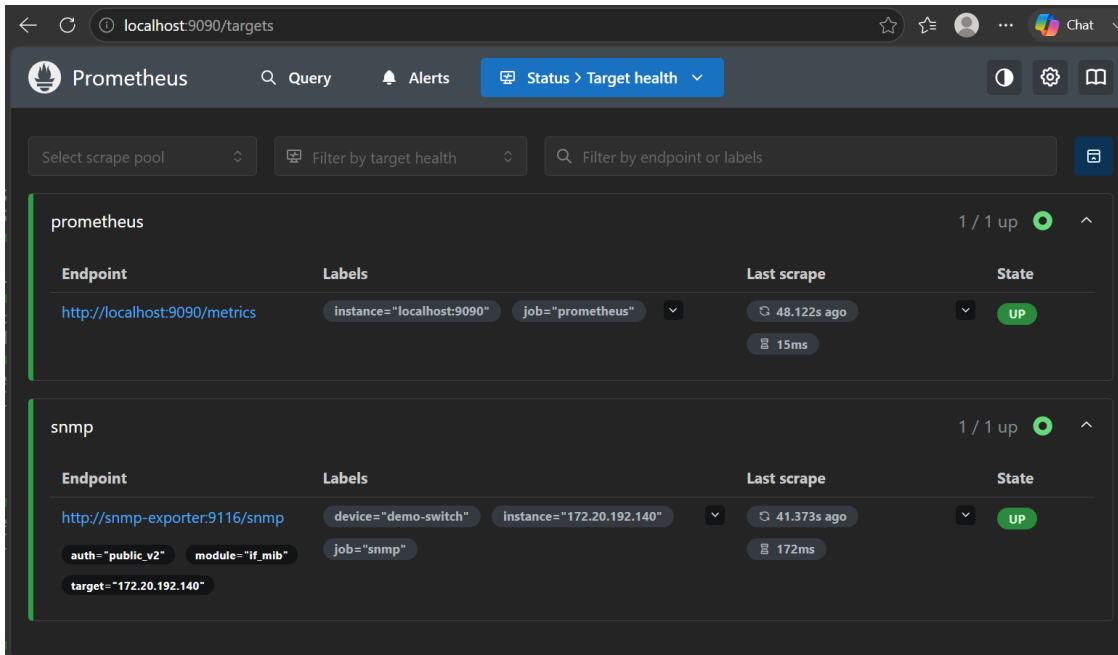
## All containers running

```
michael@Michael:~/Network monitoring stack/docker$ docker compose ps
NAME                SERVICE          IMAGE               CREATED             STATUS              PORTS
grafana             grafana          grafana/grafana:latest 4 minutes ago      Up 4 minutes        0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
prometheus          prometheus       prom/prometheus:latest 4 minutes ago      Up 4 minutes        0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
snmp-exporter       snmp-exporter    prom/snmp-exporter:latest 4 minutes ago      Up 4 minutes        0.0.0.0:9116->9116/tcp, [::]:9116->9116/tcp
```

*docker compose ps: Grafana (:3000), Prometheus (:9090), and SNMP Exporter (:9116) all showing Up status*

## 6. Prometheus — SNMP Target UP

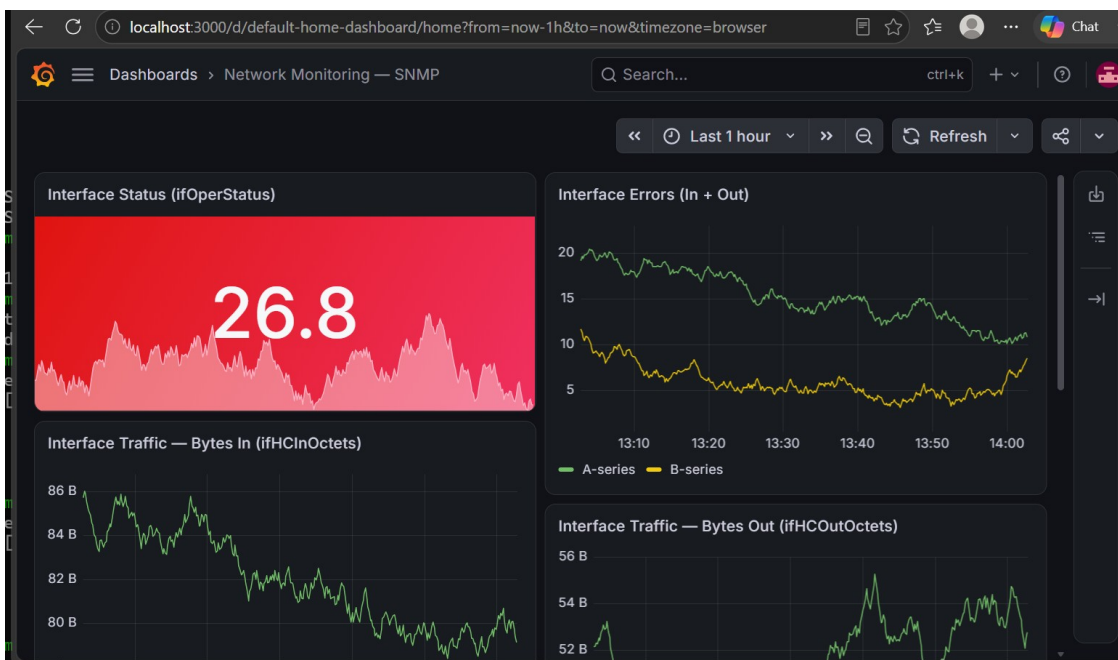
Prometheus is scraping two targets: itself (for internal metrics) and the SNMP Exporter polling the WSL host's snmpd daemon at 172.20.192.140. Both show UP with successful scrapes every 30–60 seconds.



Prometheus targets page: both prometheus (self) and snmp (172.20.192.140 via SNMP Exporter) showing UP with recent successful scrapes

## 7. Grafana — Live Network Dashboard

The pre-built Network Monitoring — SNMP dashboard auto-loaded as the Grafana home page. It displays live data pulled from Prometheus: interface operational status (ifOperStatus), inbound traffic (ifHCInOctets), outbound traffic (ifHCOutOctets), and interface error rates — all updating in real time from the SNMP Exporter's polls against the demo target.



*Grafana dashboard: Interface Status panel (red = value 26.8 from ifOperStatus aggregation), Interface Errors chart (in/out over time), Interface Traffic Bytes In and Bytes Out — all showing live, updating SNMP data*

## 8. Conclusion

The full monitoring pipeline is verified end-to-end across both platforms. On the device side, Cisco IOS routers are configured with SNMP communities and syslog forwarding, and the syslog server successfully captures real-time network events (interface state changes). On the monitoring side, Prometheus scrapes SNMP metrics from a live target via the SNMP Exporter, and Grafana renders those metrics on a pre-built dashboard with interface status, traffic, and error panels — all displaying live, updating data. In production, these two halves connect directly: Prometheus points at the real router IPs using the same SNMP communities configured in the Packet Tracer portion, and the Grafana dashboard becomes the network operations team's real-time visibility into the entire infrastructure.